Multi-Node OpenStack Cloud: Kolla-Ansible Deployment & Heat Orchestration

Mohsin Rasul

Track: Cloud

Date: August 6, 2026

Abstract

This report outlines the architecture, deployment, and validation of a persistent multi-node OpenStack private cloud environment established via Kolla-Ansible on Ubuntu Linux. Key focus areas include:

- **Containerized Architecture:** Deploying microservices across segregated control and compute nodes managed via Docker containers.
- **Infrastructure Automation:** Utilizing declarative OpenStack Heat orchestration templates (`heatStack.yaml`) for single-command provisioning.
- **Hypervisor Optimization:** Resolving nested virtualization constraints by transitioning Nova compute scheduling to QEMU emulation.

The resulting infrastructure delivers a reliable, reproducible, and persistent foundation for cloud-native workload management.

1 Introduction

This project details the setup of a multi-node OpenStack cloud across two dedicated Linux systems (`ubuntu-node1` as control and `ubuntu-node2` as compute). Moving beyond single-node sandbox tools, this deployment establishes a production-grade, container-backed environment. Core objectives include setting up virtual runtime dependencies, bootstrapping multi-node inventories, deploying OpenStack microservices, and automating infrastructure provisioning through Heat.

2 Procedures and Observations

The deployment followed a structured, sequential workflow:

- **Environment Preparation:** Installed build tools and configured an isolated Python virtual environment (`~/kolla-venv`) on the control node.
- **Inventory Configuration:** Defined host mappings in the `multinode` file, designating control and compute IP addresses alongside global deployment parameters (`/etc/kolla/globals.yml`) for networking and high availability.
- **Cluster Bootstrapping:** Executed `kolla-ansible bootstrap-servers` and `kolla-ansible deploy` to configure target hosts and launch core OpenStack containers.
- **Image and Flavor Setup:** Uploaded the CirrOS disk image to Glance and established a low-resource compute flavor (`m1.tiny`) tailored to compute node capacity.
- **Template Orchestration:** Authored a declarative Heat template (`heatStack.yaml`) to automate network, subnet, security group, and virtual machine creation.
- **Hypervisor Tuning:** Resolved initial scheduling failures (`NoValidHost`) by updating the Nova compute driver type to software emulation (`qemu`) within the global configuration parameters.

3 Summary of Results

The multi-node cloud successfully initialized containerized microservices across the cluster. The Heat orchestration engine completed the infrastructure build without manual intervention, achieving a verified `CREATE_COMPLETE` status. Verification via the Horizon web dashboard and CLI confirmed that the virtual instance (`heat-test-vm`) deployed successfully on the compute node.

4 Calculations and Graphs

The orchestrated tenant subnet was defined using a `/24` Classless Inter-Domain Routing (CIDR) block, yielding a total IP address space calculated as:

$$TotalIPAddresses = 2^{(32-24)} = 256 \tag{1}$$

Reserving network, broadcast, and gateway addresses leaves 253 usable IPs for future instance allocations. Compute resource allocation for the deployed instance relative to host capacity is expressed as:

$$MemoryAllocationRatio = \frac{512MB}{4096MB} = 12.5\% \tag{2}$$

5 Analysis of Data and Errors

Deployment challenges and their resolutions are categorized below:

- **Systematic Error:** Nested virtualization limitations on the virtualized hypervisor blocked hardware-accelerated KVM scheduling. This was systematically resolved by switching Nova's virtualization driver to QEMU emulation.
- **Configuration Error:** Initial orchestration attempts failed due to missing Heat client packages and disabled service flags. These were resolved by enabling Heat in `globals.yml` and performing targeted Ansible tag deployments.

6 Discussion, Interpretation

- **Summary of Results:** The multi-node cloud was successfully deployed, configured, and verified through automated Heat templates.
- **Significance of Findings:** Containerized service management ensures configuration persistence across host reboots, while Infrastructure as Code (IaC) eliminates manual provisioning errors.
- **Difficulties Encountered:** Hypervisor constraints required software emulation tuning to ensure reliable container execution.
- **Future Improvements:** Future work will incorporate persistent block storage (Cinder) and external router integrations for public network access.

7 Conclusion

The multi-node OpenStack private cloud was successfully provisioned and verified. Automated orchestration using Heat templates proved effective for streamlined, repeatable cloud infrastructure deployment.

References

1. OpenStack Foundation. *Kolla-Ansible Documentation*. https://docs.openstack.org/kolla-ansible/latest/

2. OpenStack Foundation. *Heat Orchestration Template (HOT) Specification*. https://docs.openstack.org/heat/latest/

3. OpenStack Foundation. *OpenStack Command-Line Interface Reference*. https://docs.openstack.org/python-openstackclient/latest/

Appendices

Appendix 1: Horizon Dashboard & Stack Verification

The following telemetry extracted from the OpenStack Horizon dashboard confirms stack creation, network topology, and quota compliance across the multi-node environment.



Figure 1: Horizon Limit Summary confirming cloud quota capacity and active service status across the multi-node cluster.

```
 GNU nano 8.7.1                                    heatStack.yaml
description: Mini-Project 2 Full Stack Deployment (Network, Subnet, Security Group, and Instance).

resources:
  # 1. Private Network
  project_network:
    type: OS::Neutron::Net
    properties:
      name: heat-private-net

  # 2. Private Subnet
  project_subnet:
    type: OS::Neutron::Subnet
    properties:
      name: heat-private-subnet
      network: { get_resource: project_network }
      cidr: 192.168.100.0/24
      dns_nameservers: [ "8.8.8.8" ]

  # 3. Security Group (Allows SSH & Ping)
  project_security_group:
    type: OS::Neutron::SecurityGroup
    properties:
      name: heat-sec-group
      description: "Allow ICMP and SSH traffic"
      rules:
        - protocol: icmp
```

Figure 2: Configuration of the declarative Heat Orchestration Template (`heatStack.yaml`) defining network, subnet, security group, and server resources.

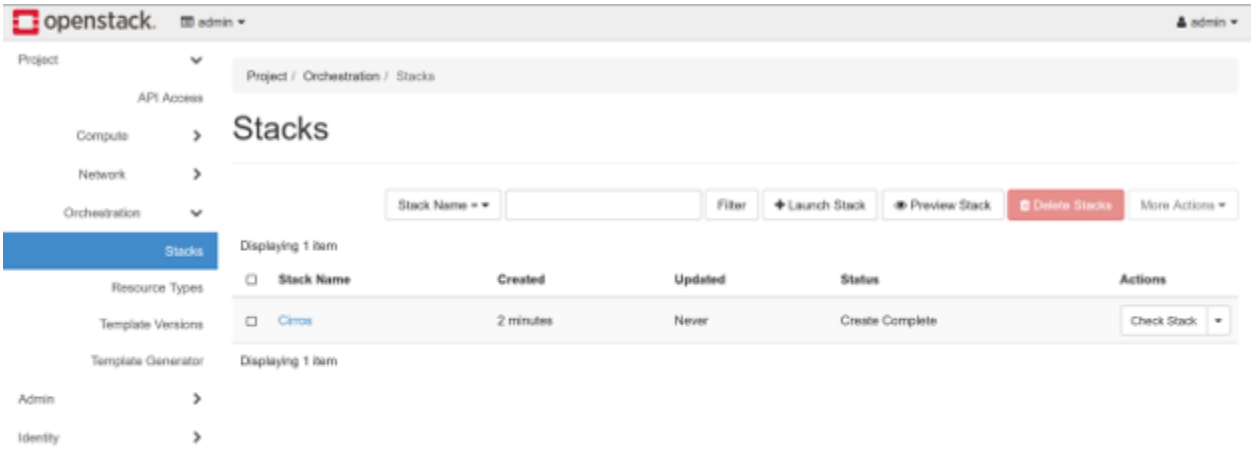


Figure 3: Horizon Orchestration Stacks view confirming the `Cirros` stack in an active `CREATE_COMPLETE` operational state.

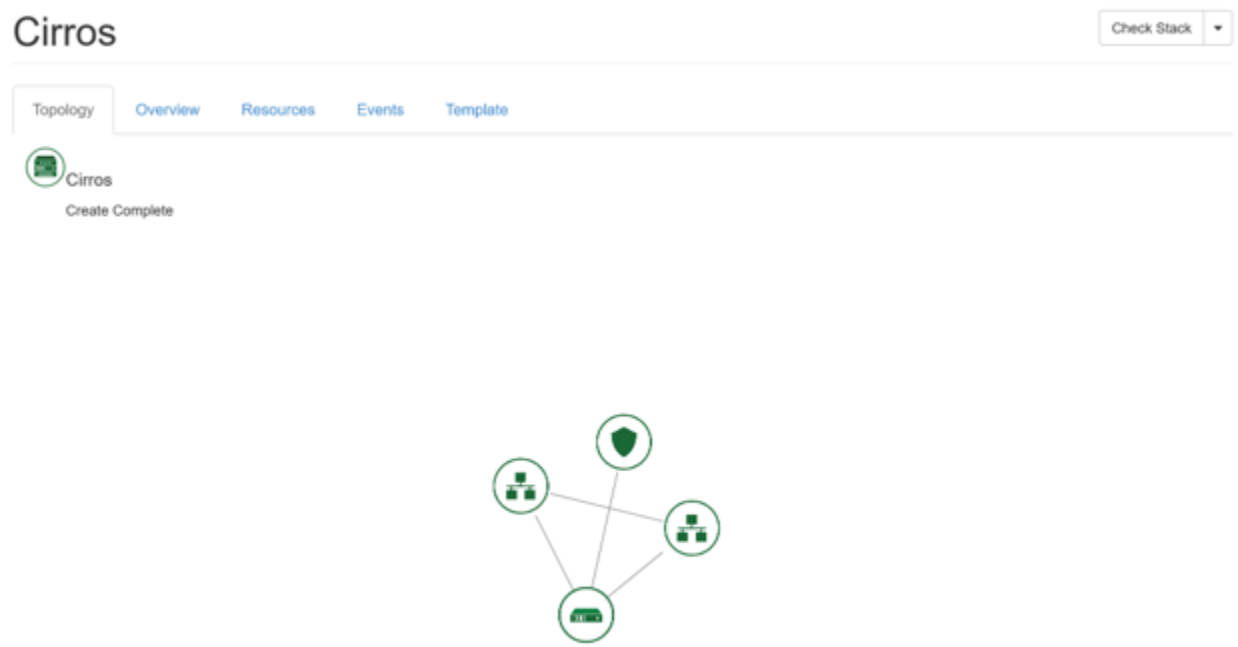


Figure 4: Visual Heat Stack Topology graph displaying active links between the deployed compute instance, security group, and private network.

Instance Name	VCPUs	Disk	RAM	Age
heat-test-vm	1	1GB	512MB	2 minutes
Displaying 1 item				

Figure 5: Horizon Instance details confirming the active `heat-test-vm` instance running on the compute node with `m1.tiny` resource constraints.

Appendix 2: Complete Bash Command History

```
 1  sudo apt update && sudo apt upgrade -y
 2  sudo apt install git python3-dev libffi-dev gcc libssl-dev
python3-venv -y
 3  python3 -m venv ~/kolla-venv
 4  source ~/kolla-venv/bin/activate
 5  pip install -U pip
 6  pip install 'ansible>=8,<9' kolla-ansible
 7  ansible -i ~/multinode all -m ping
 8  kolla-ansible bootstrap-servers -i ~/multinode
 9  kolla-ansible deploy -i ~/multinode
10  kolla-ansible post-deploy -i ~/multinode
11  pip install python-openstackclient
12  source /etc/kolla/admin-openrc.sh
13  openstack service list
14  cat /etc/kolla/passwords.yml | grep keystone_admin_password
15  nano heatStack.yaml
16  openstack flavor create --ram 512 --disk 1 --vcpus 1 m1.tiny
17  wget http://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-
disk.img
18  openstack image create "cirros" --file cirros-0.6.2-x86_64-disk.
img --disk-format qcow2 --container-format bare --public
19  openstack image list
20  openstack stack create -t heatStack.yaml Cirros
21  pip install python-heatclient
22  sudo nano /etc/kolla/globals.yml
23  kolla-ansible deploy -i ~/multinode --tags heat
24  source /etc/kolla/admin-openrc.sh
25  openstack stack create -t heatStack.yaml Cirros
26  openstack stack list
27  openstack stack show Cirros
28  openstack stack resource list Cirros
29  sudo nano /etc/kolla/globals.yml
30  kolla-ansible reconfigure -i ~/multinode --tags nova
31  openstack compute service list
32  openstack stack delete Cirros
33  openstack stack create -t heatStack.yaml Cirros
34  openstack stack list
```

```
35  openstack server list
```